# Pre-Reading: What Is a Priority Queue?

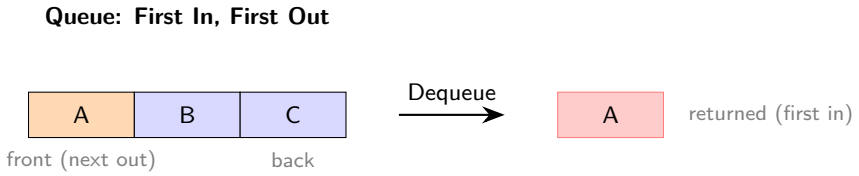
**Read this before starting P3.** It takes about 10 minutes.

In P2, you used a **queue** (FIFO) to drive BFS. The queue processed nodes in the order they were discovered — fair, but blind. This week, you'll use a **priority queue** — a data structure that always hands you the *most important* item first, regardless of when it arrived. This is the key ingredient that gives greedy best-first search its sense of direction.

---

## Quick recap: Queue (FIFO)

In P2, your BFS frontier was a **queue**. Items come out in the order they went in:

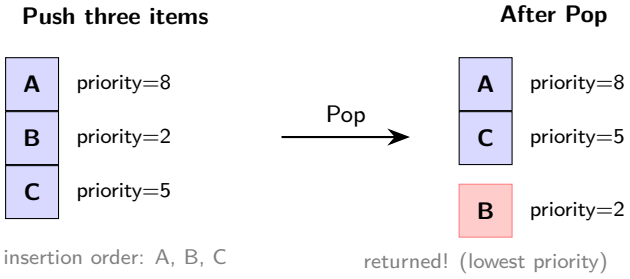



The queue doesn't care *how important* an item is — it only cares *when* it arrived. BFS treats all nodes equally.

---

## Priority Queue: Most Important First

A **priority queue** is different. Each item has a **priority** (a number). When you remove an item, you always get the one with the **lowest priority number** — the most urgent item — regardless of insertion order.




**Key insight:** Even though B was pushed *second*, it comes out *first* because it has the lowest priority (2). The priority queue doesn't care about insertion order — it cares about priority.

**Operations:**

- `push(item, priority)` — add an item with a priority number
- `pop()` — remove and return the item with the **lowest** priority number
- `is_empty()` — check if the priority queue has no items

---

## What if two items have the *same* priority?

Suppose you push C (priority=2), then later push E (priority=2). Both are equally "promising" — so which comes out first?

**The one you pushed first.** Our `PriorityQueue` breaks ties by **insertion order**: among items with equal priority, the earliest-inserted one pops first. So C (pushed first) comes out before E.

This means a priority queue behaves *exactly* like a plain FIFO queue in the special case where all priorities are equal. Priority is the **main** rule; first-in-first-out is the **tie-breaker**.

Two things to know:

- **It keeps your search reproducible.** Because ties are broken the same way every time, everyone's trace matches — as long as you push a node's neighbours in the order the grid hands them to you.
- **It doesn't change what greedy search "should" do.** Two equally-promising nodes are, by definition, equally good next steps. Picking either is a valid greedy move; the tie-break just makes the choice deterministic.

  **Note:** `push` and `pop` are just generic names for "add" and "remove" — they do **not** make this a stack. A stack's `pop` returns the *most recently* added item (LIFO); our `pop` returns the item with the *lowest priority*. Same method names, completely different behaviour.

---

## Comparing the three data structures

You've now seen three data structures across P1–P3. Each one determines the order items come out:

| Stack (LIFO) | Queue (FIFO) | Priority Queue |
|---|---|---|
| Last in, first out | First in, first out | Lowest priority first |
| DFS | BFS | Greedy best-first |
| Dives deep | Layer by layer | Toward the goal |

**The algorithm structure is the same every time.** The only thing that changes is the frontier data structure. That one choice completely changes the search behaviour:

- **Stack** → DFS: dives deep into one path before backtracking
- **Queue** → BFS: explores layer by layer, guaranteed shortest path
- **Priority queue** → Greedy best-first: always expands the most promising node

---

## How it works in Python

We provide a `PriorityQueue` class in `p3_helpers.py`. You don't need to know how it works internally — just use these three methods:

```
from p3_helpers import PriorityQueue

pq = PriorityQueue()

# Push items with priorities
pq.push("far away", priority=10)
pq.push("very close", priority=2)
pq.push("medium", priority=5)

# Pop always returns the lowest-priority item
print(pq.pop())   # "very close" (priority=2)
print(pq.pop())   # "medium" (priority=5)
print(pq.pop())   # "far away" (priority=10)
```

**Compare with the Queue from P2:**

| | Queue (P2) | Priority Queue (P3) |
|---|---|---|
| Add an item | `enqueue(item)` | `push(item, priority)` |
| Remove an item | `dequeue()` | `pop()` |
| Which item comes out? | The oldest (FIFO) | The lowest priority |
| Check if empty | `is_empty()` | `is_empty()` |

The only real difference: `push` takes a **priority** argument. Everything else is the same pattern you already know.

---

## Quick self-test

Before moving on, make sure you can answer these:

1. You push A (priority=5), B (priority=1), C (priority=3) into a priority queue. You pop twice. What do you get?
2. In greedy best-first search, what do we use as the priority for each node?
3. If two nodes have priorities 4 and 7, which one gets expanded first?

**Answers:** (1) B (priority=1), then C (priority=3). (2) The heuristic estimate of distance to the goal. (3) The node with priority 4 (lower number = higher priority).

---

**Want to learn more?** Priority queues are typically implemented using a data structure called a **heap** — a special kind of binary tree that keeps the smallest element at the root. You'll learn about heaps in **FIT1008 Introduction to Computer Science**. Python's `heapq` module provides a heap implementation, which is what our `PriorityQueue` class uses internally.

---